

Choose a GPT-6 model based on the reasoning your task requires, speed, and cost.



GPT-6 Astra

Highest intelligence
For the most demanding reasoning, coding, and professional work.



GPT-6.1 Sol

Balanced speed, cost, and intelligence
Near-Astra performance for complex work at a lower cost.



GPT-6 Luna

Fastest and most cost-effective
Strong performance for focused, high-volume tasks.

To get started, set `model` in a [Responses API](#) request. If you already use `gpt-6-sol`, review the [migration guidance](#) before switching to GPT-6.1 Sol.

GPT-6 Astra

GPT-6 Astra is our most intelligent model yet, with state-of-the-art performance in computer use, browsing, software engineering, science, and professional work. It can carry out multi-step workflows across code, browsers, and professional software. In [several evaluations](#), Astra achieved stronger results using substantially fewer output tokens. Its estimated API cost per task was lower than earlier models despite its higher per-token pricing.

GPT-6 Astra is also our most aligned model yet. It excels at exercising care, respecting task boundaries, and communicating transparently. When instructions leave room for interpretation, it uses the context it has to fill in routine gaps and asks focused questions when the answer could change the outcome. It incorporates new requirements, changes course when asked, and answers side questions without losing track of the broader task.

All GPT-6 Astra users also have access to [Fast mode](#) and the new [Ultrafast mode](#) for our fastest API speeds.

GPT-6.1 Sol

Use GPT-6.1 Sol for complex coding, computer use, and professional work when you want near-Astra performance at a lower cost. Compare it with Astra on your task to assess the tradeoff between quality and cost.

Ask AI

Loading docs agent...

Astra, our systems asynchronously monitor for misalignment and trigger alerts when necessary. See [Misalignment monitoring](#) for more information.

GPT-6 also supports the existing API capabilities available with GPT-5.6, including [computer use](#), [Structured Outputs](#), [streaming](#), [Programmatic Tool Calling](#), [multi-agent orchestration](#), [prompt caching](#), [persisted reasoning](#), [compaction](#), and [pro mode](#).

Limitations

- GPT-6 Astra and GPT-6.1 Sol do not support the `none` reasoning effort; GPT-6 Sol and GPT-6 Luna do.
- Fast mode supports EU data residency for GPT-6.1 Sol, GPT-6 Sol, and GPT-6 Luna, but not GPT-6 Astra. [Ultrafast mode](#) for GPT-6.1 Sol supports US and EU

Loading docs agent...

testing before considering a task complete. For smaller tasks, this can result in broader tests than the task requires.

Initiative and follow-through

GPT-6 Astra is generally better than GPT-5.6 Sol and earlier models at staying coherent during long tasks. It is also more likely to ask for clarification where earlier models would make assumptions.

To encourage more autonomous work, start with this prompt:

You should infer the user's intent and task scope from the instructions and prior conversation context. Your job is to bias towards action and carry the user's intended task to completion.



When the user expresses intent to perform new work or fix an

Loading docs agent...

...before approving a change, releasing a feature, for example, before deploying a change, writing to an external application, merging a PR or publishing a site, do all the required work first so that user approval is the final step. You don't need user permission for reversible tasks, read-only actions, reviews or fixes, or anything for which authorization is provided earlier in the session or strongly implied from the task instruction.

Do not introduce unsolicited warnings, disclaimers, approval flows, or safety/compliance checklists due to hypothetical risk.

The model also likes to ask non-blocking questions as it's working by default, so adjust these prompts to match the level of autonomy your application needs.

Instruction following

Loading docs agent...

Default to using clear, concise paragraphs, each developing one main idea. Use lists only when the information is genuinely parallel, sequential, or easier to compare, and avoid nested lists unless the hierarchy cannot be expressed clearly in prose. Use plain, simple language: familiar words, concrete examples, and precise verbs. Prefer active voice and direct statements.



Make sure to state the main point clearly and early, then develop it with the explanation and detail the reader needs. Let each sentence build on what came before. Develop the points that matter and provide enough support to be useful.

For technical communication, the following prompt helps strike a balance between using clear, coherent language while remaining domain appropriate:

Loading docs agent...

Following prompt to tune how much GPT-5 Astra should delegate work:

If at any point you can parallelize work by delegating tasks to another agent (no matter if you are the root or subagent), you should do so using collaboration tools if it could save time or improve quality.



Messages between agents may contain grammar or spacing errors. Use this prompt to make inter-agent messages easier to read:

Messages that you send to other agents and your final answer may be read by a human, so ensure they are legible. Always put proper spaces between words and/or numbers.



Loading docs agent...

To use this skill in other coding agents, download it from the [Codex repository](#).

Update API and model parameters

Set `model` to `gpt-6-astra`, `gpt-6.1-sol`, or `gpt-6-luna`, then check the following:

- **Reasoning effort:** Preserve your current effective reasoning effort where supported. GPT-6 Astra and GPT-6.1 Sol do not support `none`; use `low` instead. GPT-6 Sol and GPT-6 Luna support `none`. If your existing request uses `minimal`, start with `low` and compare results on representative tasks. Use `reasoning.effort` in Responses or `reasoning_effort` in Chat Completions.
- **Tool calling:** Use the [Responses API](#). GPT-6 Astra and GPT-6.1 Sol support Chat Completions, but tool calling requires Responses. GPT-6 Sol and GPT-6 Luna

Loading docs agent...

Previous models

GPT-5.6

GPT-5.5

GPT-5.4

GPT-5.3 Codex

GPT-5.2

GPT-5.1

GPT-5

GPT-4.1

Introduction

GPT-5.6 sets a new quality and efficiency baseline for complex production workflows. GPT-5.6 is especially token-efficient and improves frontend aesthetics, including layout, visual hierarchy, and design judgment.

GPT-5.6 also introduces a new naming scheme. The `gpt-5.6` alias routes requests to `gpt-5.6-sol`, the model for flagship capability. Use `gpt-5.6-terra` for strong

Loading docs agent...

select the behavior. Learn how to [preserve reasoning across calls](#).

- **Max reasoning effort:** GPT-5.6 supports `max` reasoning effort for demanding tasks that need more exploration and verification. If you currently use `xhigh`, compare both settings on representative workloads.
- **Pro mode:** GPT-5.6 can perform more model work to improve reliability on difficult tasks and return a single final answer. Enable it with `reasoning.mode: "pro"` when quality matters more than latency and token usage. Learn how to [use pro mode](#).
- **Token efficiency:** GPT-5.6 reaches flagship-level performance with fewer output tokens.
- **Frontend design:** GPT-5.6 creates more polished and usable websites and applications, with stronger layout, visual hierarchy, and design judgment.
- **Intent understanding:** GPT-5.6 can better infer the user's underlying goal and intended level of work from context, so you often do not need to prescribe every

Loading docs agent...

Migration quickstart

Migrate with Codex

Codex can apply the recommended changes in this guide with the [OpenAI Docs skill](#).

```
$openai-docs migrate this project to the GPT-5.6 model family
```



To use this skill in other coding agents, download it from the [OpenAI skills repository](#).

Update API and model parameters

- Choose the target model for the workload. Use `gpt-5.6-sol` for flagship capability, `gpt-5.6-terra` for a balance of intelligence and cost, or `gpt-5.6-`

Loading docs agent...

- Omit `reasoning.context` or set it to `auto` to use `all_turns`, the GPT-5.6 default. Check the response's `reasoning.context` field to confirm the effective mode.
- Set `reasoning.context` to `all_turns` when the task's goals, assumptions, and priorities stay stable across turns.
- With `all_turns`, continue with `previous_response_id` to make reasoning from earlier responses available to the model.
- When managing history manually, preserve and resend previous user inputs and every response output item. For `store: false` or Zero Data Retention, replay the encrypted reasoning items that the API returns by default.
- Set `reasoning.context` to `current_turn` when earlier reasoning is no longer relevant.

Loading docs agent...

representative tasks from your own application.

To simplify prompts without losing important guidance:

- Start with a prompt and tool set that already works. Remove one group of instructions, examples, or tools at a time, then rerun the same evals.
- State each instruction once.
- Expose only tools relevant to the task, and keep their descriptions concise and precise.
- Keep examples and style guidance when they encode a product requirement or correct a measured gap.
- Track context both at the start of a run and as the conversation grows. Long sessions can amplify repeated prompt and tool content.

Define autonomy and approval boundaries

Loading docs agent...

GPT-5.6 tends to be more concise by default than GPT-5.5. When migrating, check whether broad brevity instructions such as “Be concise” or “Keep it short” are still useful. They may be unnecessary for some tasks and can sometimes make responses too brief. Keep them when they reliably produce the output your application needs.

For more consistent control across requests, use `text.verbosity` to set the default level of detail, then use the prompt for task-specific requirements.

Set a default with `text.verbosity`

Choose `low`, `medium`, or `high` as the default level of detail for a request. In the prompt, specify any task-specific length, structure, or required content. See [Set up `text.verbosity`](#) for an API example.

Loading docs agent...

relevant. Omit generic praise and unnecessary sign-offs.

Pro mode

Choose pro mode when quality matters most

Pro mode is a Responses API execution mode that applies more model work to a request before returning a single final answer. It can improve reliability on difficult tasks, but it increases latency and aggregates the tokens from that work in reported usage. Those tokens are billed at the selected model's standard token rates.

Use pro mode when a marginal quality improvement materially affects the outcome and the task is difficult enough to benefit, such as complex optimization, high-value coding or review, or deep analysis with clear evaluation criteria. Prefer standard mode

Loading docs agent...

Compare standard and pro modes on the same representative tasks. Measure task success, answer completeness, required evidence, total tokens, latency, and cost. Use pro mode selectively where its quality or reliability gain justifies the extra model work.

Learn more in the [reasoning mode guide](#).

Programmatic Tool Calling

Choose Programmatic Tool Calling by task shape

Programmatic Tool Calling (PTC) works best for bounded workflows where code can process several tool results or large intermediate outputs and return a much smaller structured result. Use it for filtering, joining, ranking, deduplication, aggregation, validation, or other predictable processing.

Loading docs agent...

For example:

<tool_orchestration>

Use Programmatic Tool Calling for [bounded stage] using only [eligible tools].

Run independent calls concurrently when safe. Use only documented tool input and output fields.

Process and reduce the intermediate results, then emit exactly [output schema], including the evidence needed for the final answer.

Stop when [condition] is met. Retry transient failures at most [R] times.

Do not repeat completed calls or perform side-effecting actions.



Loading docs agent...